PRESSURE DETECTION

Mastering Embedded System Diploma

Design & Implementation Report

Author

Eng. Adel Shata

Embedded Systems
Software Engineering

GitHub: github.com/Adel-Shata
LinkedIn: linkedin.com/in/adel-shata-a1b9a7376
Email: adel.shata.eng@gmail.com

August 2026

1. Overview

A bare-metal embedded C project targeting an STM32F103C6 (ARM Cortex-M3) microcontroller, simulated in Proteus VSM. The system monitors cabin pressure and triggers an alarm when a configurable threshold is exceeded, following SysML/UML modelling done in TTool.

The software is decomposed into four independent blocks, each implemented as a separate .c/.h module with file-scope (static) state and explicit interface functions — mirroring the AVATAR/TTool block diagram signals. The project includes a custom linker script and startup file written from scratch (no HAL, no CMSIS).

2. Case Study

Scenario: Monitor aircraft cabin pressure and alert the crew when pressure exceeds safe limits.

PARAMETER	VALUE
Target MCU	STM32F103C6 (ARM Cortex-M3)
Pressure threshold	20 bar
Alarm duration	60 seconds
Sensor input	GPIOA PA0–PA7 (8-bit DIP-switch bank)
Alarm output	PA13 (active-low LED)

Assumptions:

- No sensor failure modelled.
- No actuator failure modelled.
- No power-cut or brown-out scenario modelled.
- Pressure readings are instantaneous (no ADC conversion delay modelled).
- Timing is approximate (busy-wait delay loops, not timer-based).

3. System Architecture

The system is decomposed into four independent blocks, each as a separate .c/.h module:

MODULE	RESPONSIBILITY	INTERFACE FUNCTIONS
Pressure_Sensor_Driver	Reads raw pressure from hardware (GPIO)	pressureSensorInit(), pressureSensorRead()
High_Pressure_Detection	Compares reading against threshold, raises flag	highPressureDetectionfsmTick(), highPressureDetected()
Alarm	Reacts to high-pressure flag, drives alarm for 60s	alarmfsmTick()
Alarm_Actuator_Driver	Reactive actuator driver; turns alarm on/off	alarmActuatorInit(), alarmActuatorOn(), alarmActuatorOff()

Communication model:

All blocks communicate through explicit interface functions declared in their headers. There are no shared global variables between blocks — each block's internal state is kept static (file-scope only). The system runs a single super-loop that calls each block's tick function synchronously.

→ Block Diagram (TTool)

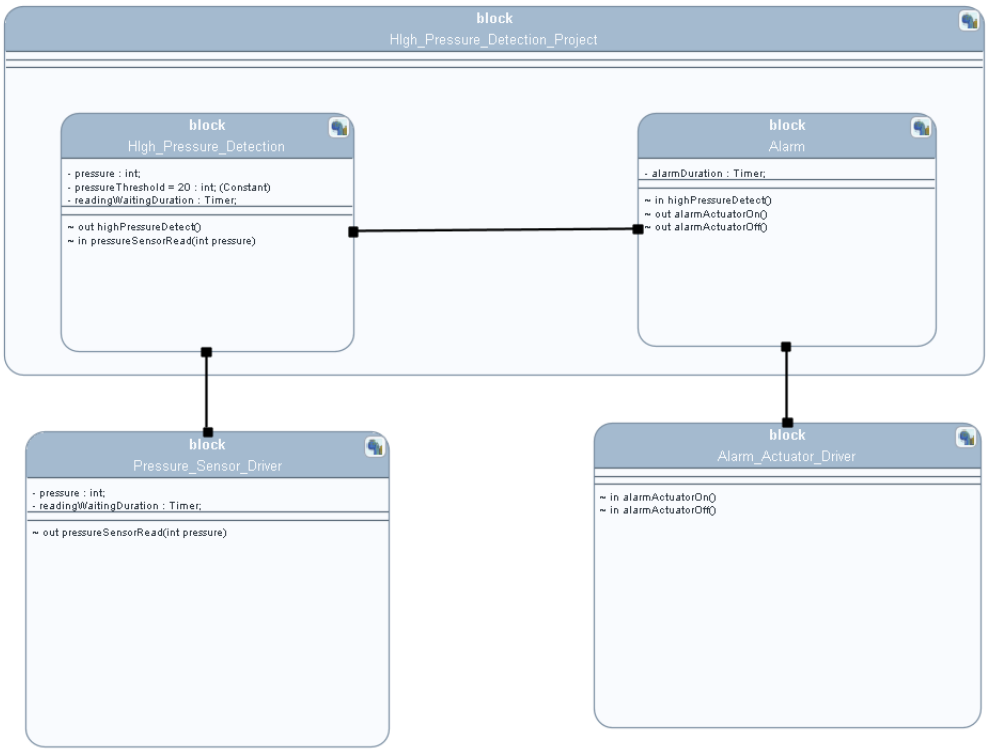

Figure 1 — System Block Diagram

4. Design Documentation (TTool / SysML / UML)

Full SysML/UML diagrams were produced in TTool and are included below ↓↓.

→ Requirement Diagram

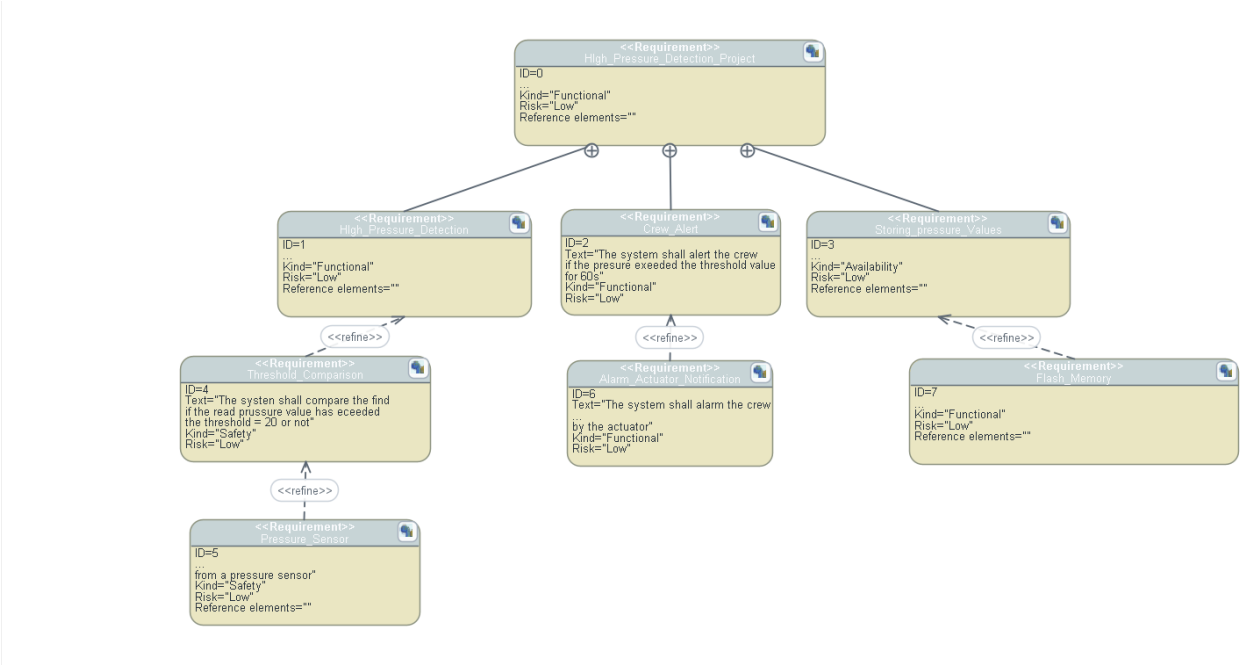

Figure 2 — Requirement Diagram

→ Use Case Diagram

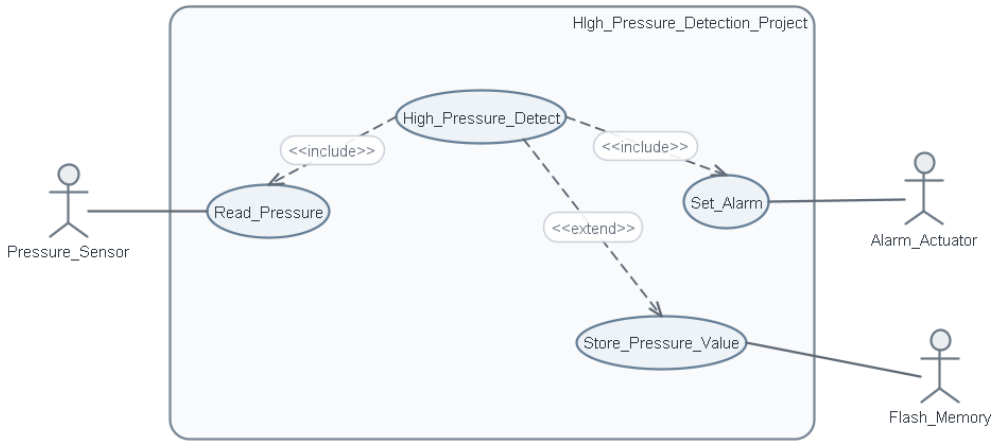

Figure 3 — Use Case Diagram

→ Activity Diagram

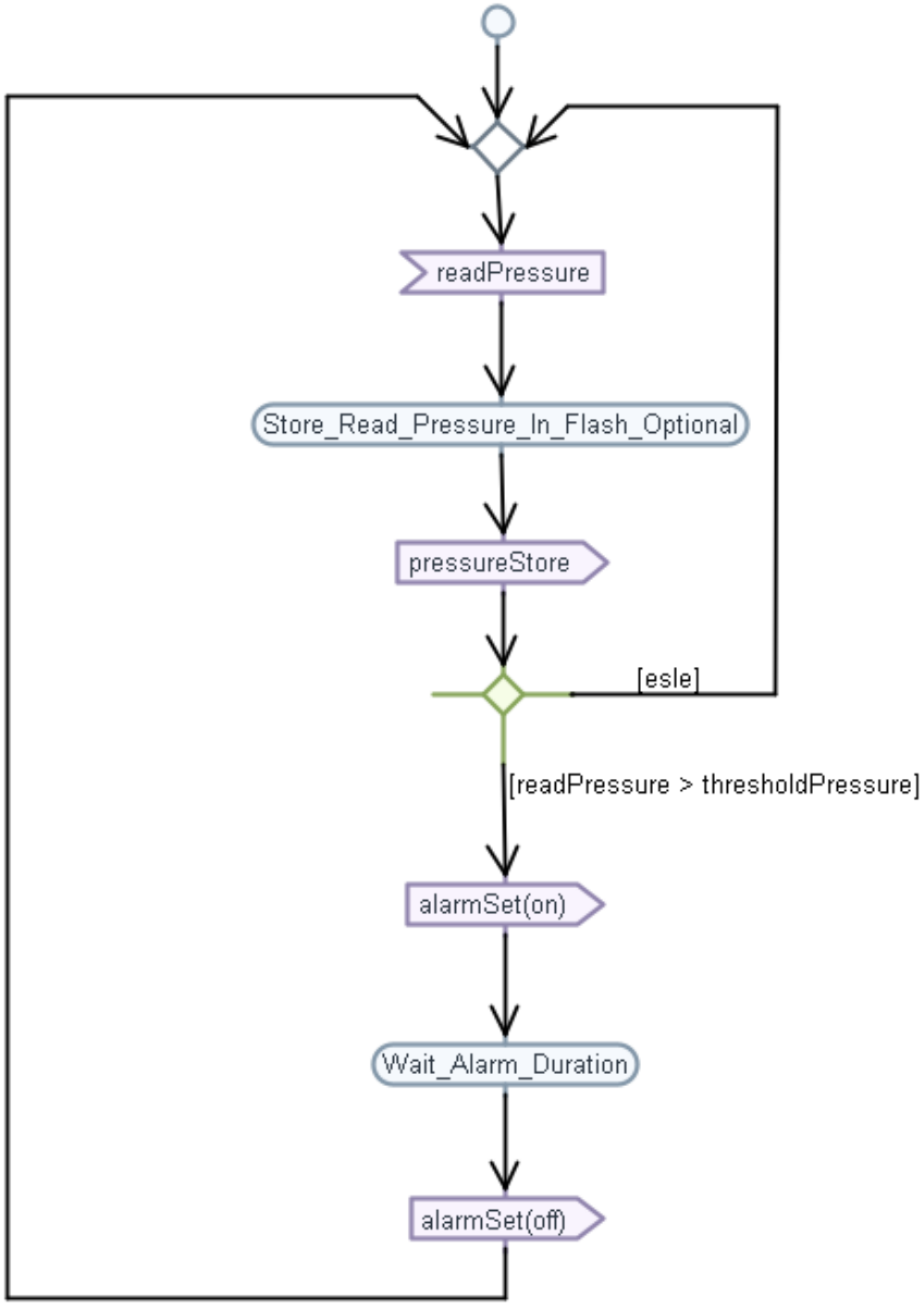

Figure 4 — Activity Diagram

→ Sequence Diagram

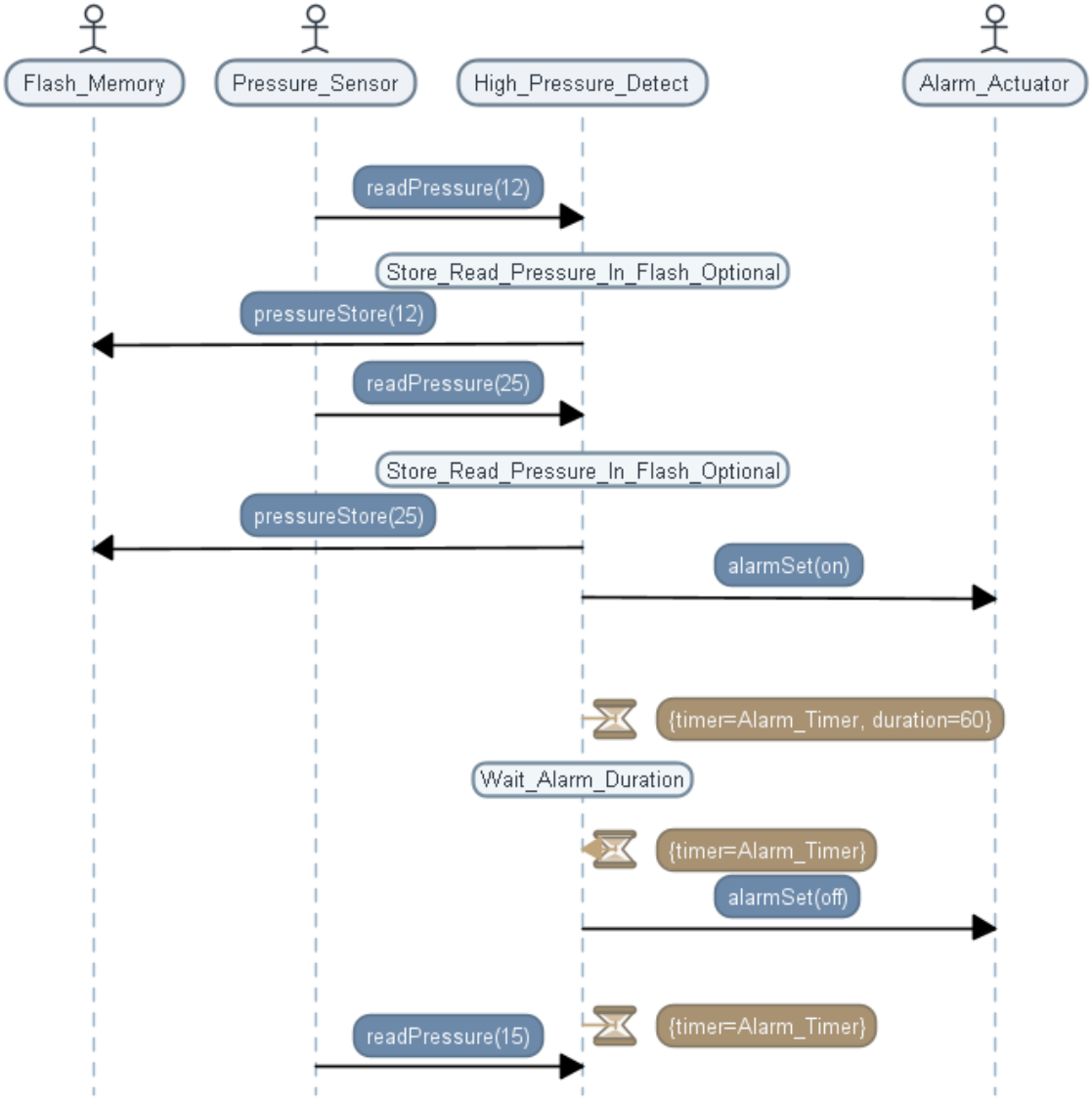

Figure 5 — Sequence Diagram

5. State Machine Diagrams

Each software block has a corresponding state machine diagram modelled in TTool.

→ Pressure Sensor Driver

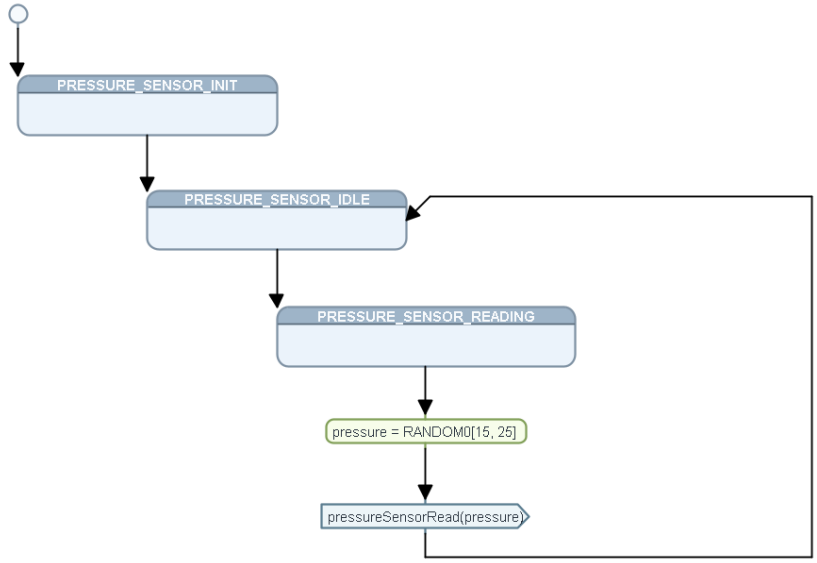

Figure 6 — State Machine — Pressure Sensor Driver

STATE	DESCRIPTION
PRESSURE_SENSOR_INIT	Initial state after pressureSensorInit()
PRESSURE_SENSOR_IDLE	Waiting for read request
PRESSURE_SENSOR_READING	Actively reading pressure value from GPIO

→ High Pressure Detection

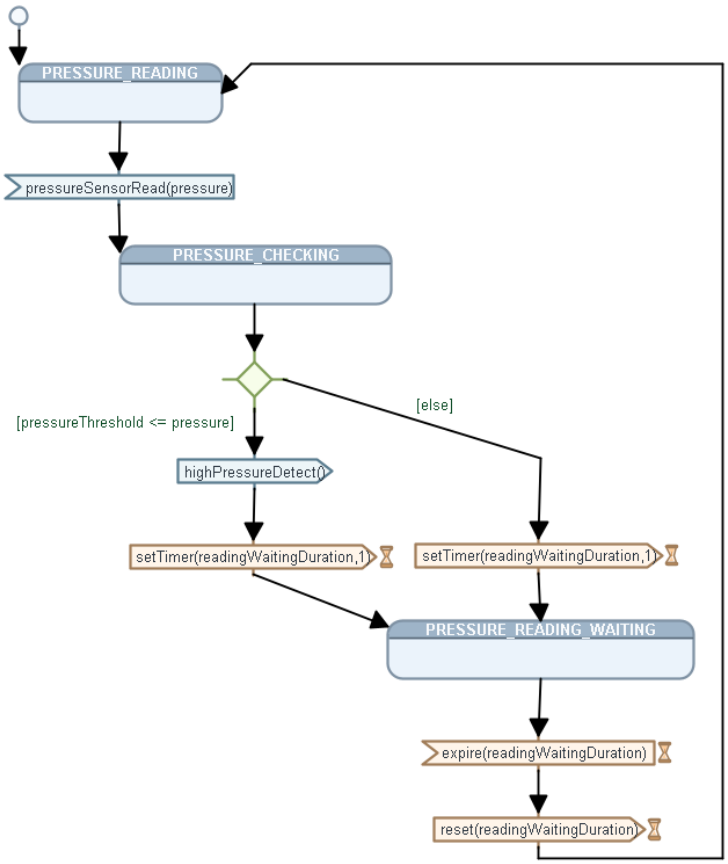

Figure 7 — State Machine — High Pressure Detection

STATE	DESCRIPTION
PRESSURE_READING	Call pressureSensorRead() to get current value
PRESSURE_CHECKING	Compare reading against HIGH_PRESSURE_THRESHOLD (20)
PRESSURE_READING_WAITING	Wait 1 second before next reading cycle

→ Alarm

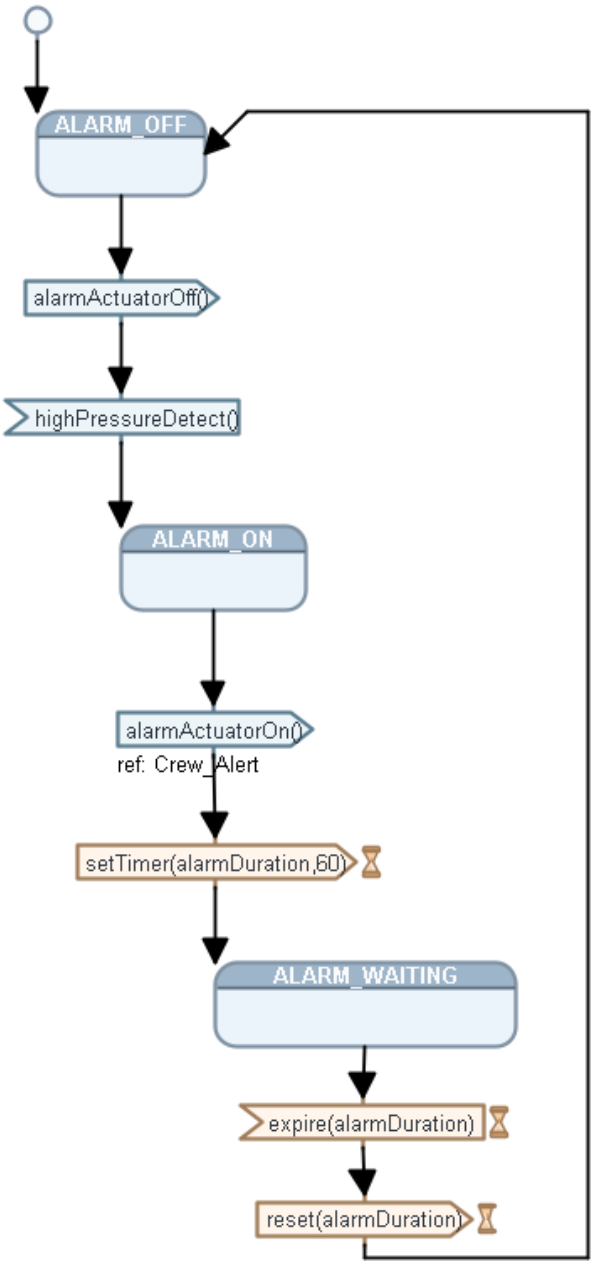

Figure 8 — State Machine — Alarm

STATE	DESCRIPTION
ALARM_OFF	Alarm inactive; check highPressureDetected()
ALARM_ON	Alarm active; call alarmActuatorOn()
ALARM_WAITING	Wait 60 seconds (delay(60000)) before turning off

→ Alarm Actuator Driver

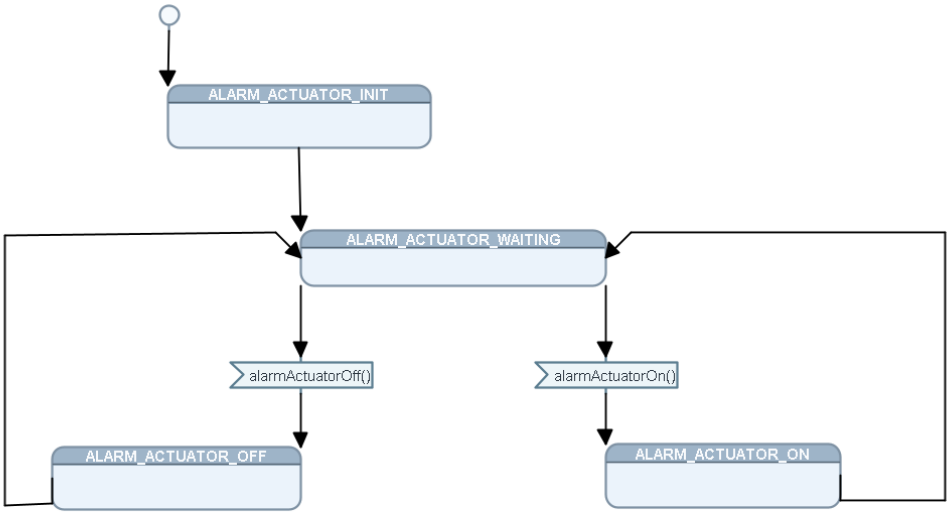

Figure 9 — State Machine — Alarm Actuator Driver

STATE	DESCRIPTION
ALARM_ACTUATOR_INIT	Initial state after alarmActuatorInit()
ALARM_ACTUATOR_WAITING	Ready to accept on/off commands
ALARM_ACTUATOR_ON	Alarm output active (PA13 LOW)
ALARM_ACTUATOR_OFF	Alarm output inactive (PA13 HIGH)

6. Source Code

The implementation follows a strict one-block-per-module structure. Every block's internal state is kept static (file-scope only), preserving encapsulation between blocks.

→ main.c

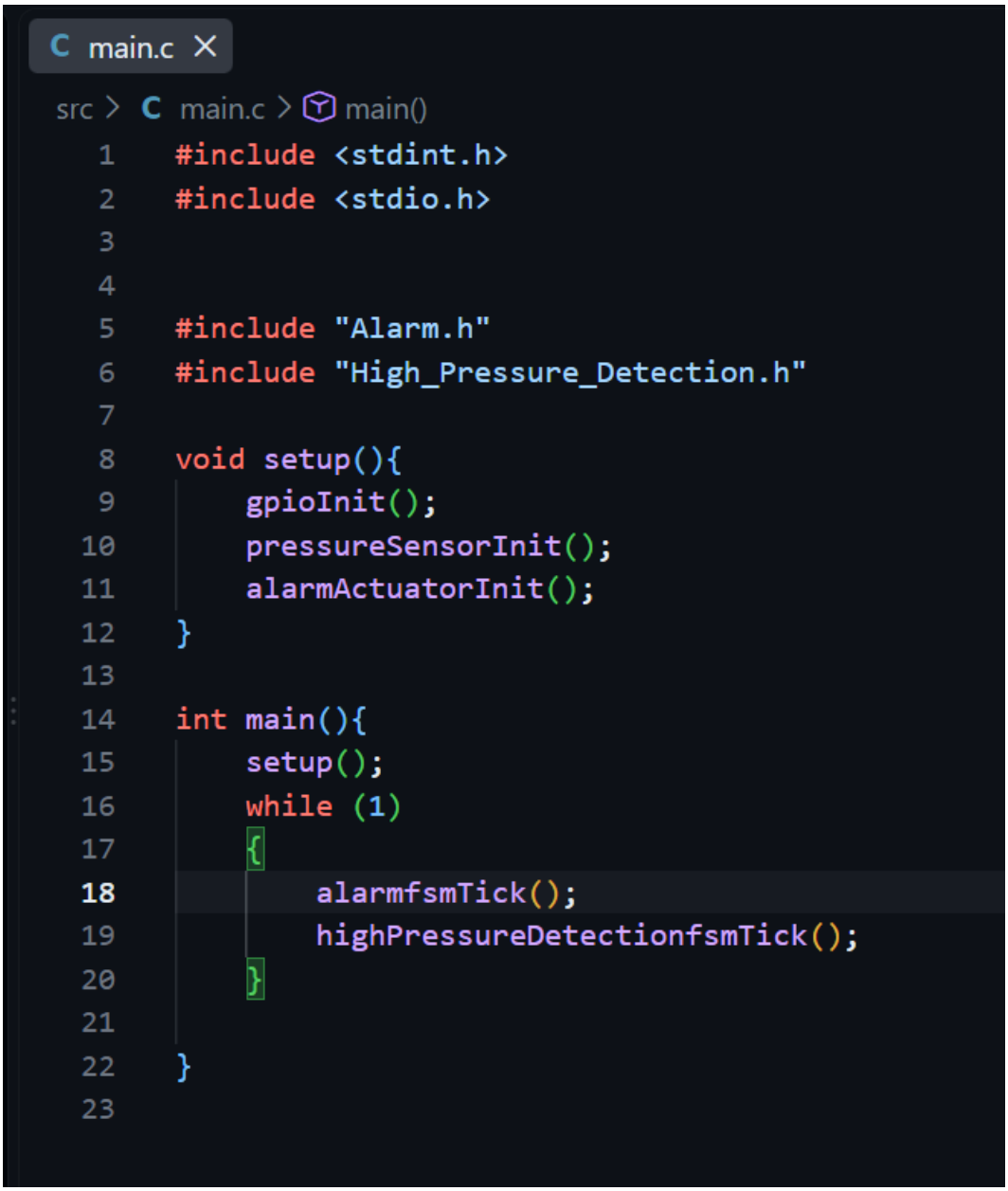

Figure 10 — main.c

The entry point. Calls setup() to initialize all modules, then runs a super-loop calling alarmfsmTick() and highPressureDetectionfsmTick() repeatedly.

→ Low-Level Driver — driver.c / driver.h

driver.c

src > driver > driver.c > setAlarmActuator(int)

```c
#include "driver/driver.h"
#include <stdint.h>
#include <stdio.h>

void delay(int nCount)
{
    for(; nCount != 0; nCount--);
}

int getPressureVal(){
    return (GPIOA_IDR & 0xFF);
}

void setAlarmActuator(int i){
    if (i == 0){
        SET_BIT(GPIOA_ODR,13);
    }
    else if (i == 1){
        RESET_BIT(GPIOA_ODR,13);
    }
}

void gpioInit(){
    SET_BIT(APB2ENR, 2);
    GPIOA_CRL &= 0xFF0FFFFF;
    GPIOA_CRL |= 0x00000000;
    GPIOA_CRH &= 0xFF0FFFFF;
    GPIOA_CRH |= 0x22222222;
}

```

Figure 11 — driver.c

driver.h

inc > driver > driver.h > ...

```c
#ifndef DRIVER_H_
#define DRIVER_H_

#include <stdint.h>
#include <stdio.h>

#define SET_BIT(ADDRESS,BIT)   ADDRESS |=  (1<<BIT)
#define RESET_BIT(ADDRESS,BIT) ADDRESS &= ~(1<<BIT)
#define TOGGLE_BIT(ADDRESS,BIT)  ADDRESS ^=  (1<<BIT)
#define READ_BIT(ADDRESS,BIT) ((ADDRESS) &   (1<<(BIT)))


#define GPIO_PORTA 0x40010800
#define BASE_RCC   0x40021000

#define APB2ENR   *(volatile uint32_t *)(BASE_RCC + 0x18)

#define GPIOA_CRL *(volatile uint32_t *)(GPIO_PORTA + 0x00)
#define GPIOA_CRH *(volatile uint32_t *)(GPIO_PORTA + 0X04)
#define GPIOA_IDR *(volatile uint32_t *)(GPIO_PORTA + 0x08)
#define GPIOA_ODR *(volatile uint32_t *)(GPIO_PORTA + 0x0C)


void delay(int nCount);
int  getPressureVal();
void setAlarmActuator(int i);
void gpioInit();

#endif // DRIVER_H_
```

Figure 12 — driver.h

Hardware abstraction layer. Provides:

- gpioInit() — configure GPIOA (enable clock, set pin modes)
- getPressureVal() — read 8-bit value from GPIOA_IDR
- setAlarmActuator(int i) — control PA13 (active-low)
- delay(int nCount) — busy-wait delay loop
- Register macros: SET_BIT, RESET_BIT, TOGGLE_BIT, READ_BIT

→ Pressure Sensor Driver — Pressure_Sensor_Driver.c / .h

Pressure_Sensor_Driver.c

src > driver > Pressure_Sensor_Driver.c > __unnamed_enum_0417_1

```c
#include "driver/Pressure_Sensor_Driver.h"

static enum{
    PRESSURE_SENSOR_IDLE = 0,
    PRESSURE_SENSOR_READING
}pressureSensorState;


void pressureSensorInit(){
    pressureSensorState = PRESSURE_SENSOR_IDLE;
}

void pressureSensorRead(int32_t *pressure){
    if(pressureSensorState == PRESSURE_SENSOR_IDLE){
        pressureSensorState = PRESSURE_SENSOR_READING;
        *pressure = getPressureVal();
    }
    pressureSensorState = PRESSURE_SENSOR_IDLE;
}
```

Figure 13 — Pressure_Sensor_Driver.c

Pressure_Sensor_Driver.h

inc > driver > Pressure_Sensor_Driver.h > ...

```c
#ifndef PRESSURE_SENSOR_DRIVER_H_
#define PRESSURE_SENSOR_DRIVER_H_

/*Section 1: Include Files*/
#include "driver.h"
/*Section 2: Macros*/
/*Section 3: Macro Functions*/
/*Section 4: Function Prototypes*/

/**
 * @brief Initialize the pressure sensor
 */
void pressureSensorInit();

/**
 * @brief Read the pressure value from the sensor
 * @param pressure Pointer to store the read pressure value
 */
void pressureSensorRead(int32_t *pressure);
#endif // PRESSURE_SENSOR_DRIVER_H_
```

Figure 14 — Pressure_Sensor_Driver.h

FSM-based driver that reads pressure from hardware. Transitions: IDLE → READING → getPressureVal() → IDLE.

→ High Pressure Detection — High_Pressure_Detection.c / .h

High_Pressure_Detection.c

src > C High_Pressure_Detection.c > highPressureDetectionfsmTick(void)

```c
#include "High_Pressure_Detection.h"

static enum{
    PRESSURE_READING,
    PRESSURE_CHECKING,
    PRESSURE_READING_WAITING
}pressureDetectionState = PRESSURE_READING;

static highPressureState_t pressureValue = PRESSURE_NOT_HIGH;
static int32_t pressure;

void highPressureDetectionfsmTick(void){
    switch(pressureDetectionState){
        case PRESSURE_READING:
            pressureSensorRead(&pressure);
            pressureDetectionState = PRESSURE_CHECKING;
            break;
        case PRESSURE_CHECKING:
            if(pressure >= HIGH_PRESSURE_THRESHOLD){
                pressureValue = PRESSURE_HIGH;
            } else {
                pressureValue = PRESSURE_NOT_HIGH;
            }
            pressureDetectionState = PRESSURE_READING_WAITING;
            break;
        case PRESSURE_READING_WAITING:
            delay(1000); // Wait 1 second before next reading
            pressureDetectionState = PRESSURE_READING;
            break;
    }
}

highPressureState_t highPressureDetected(void){
    return pressureValue;
}
```

Figure 15 — High_Pressure_Detection.c

High_Pressure_Detection.h

inc > C High_Pressure_Detection.h > HIGH_PRESSURE_THRESHOLD

```c
#ifndef HIGH_PRESSURE_DETECTION_H_
#define HIGH_PRESSURE_DETECTION_H_

/*Section 1: Include Files*/
#include "driver/Pressure_Sensor_Driver.h"
/*Section 2: User Defined Types*/
typedef enum{
    PRESSURE_NOT_HIGH,
    PRESSURE_HIGH
}highPressureState_t;
/*Section 2: Macros*/
#define HIGH_PRESSURE_THRESHOLD 20 // Example threshold value
/*Section 3: Macro Functions*/
/*Section 4: Function Prototypes*/

/**
 * @brief Finite State Machine tick function for high pressure detection
 */
void highPressureDetectionfsmTick(void);

/**
 * @brief Detects if the pressure is high
 * @return The state of the pressure detection
 */
highPressureState_t highPressureDetected(void);


#endif // HIGH_PRESSURE_DETECTION_H_
```

Figure 16 — High_Pressure_Detection.h

FSM that reads pressure, compares against threshold (20 bar), and sets the PRESSURE_HIGH / PRESSURE_NOT_HIGH flag. Includes 1-second delay between readings.

→ Alarm — Alarm.c / .h

Alarm.c

src > Alarm.c > alarmfsmTick(void)

```c
#include "Alarm.h"

static enum{
    ALARM_OFF,
    ALARM_ON,
    ALARM_WAITING
}alarmState = ALARM_OFF;


void alarmfsmTick(void) {
    switch(alarmState){
        case ALARM_OFF:
            alarmActuatorOff();
            if(highPressureDetected() == PRESSURE_HIGH){
                alarmState = ALARM_ON;
            }
            break;
        case ALARM_ON:
            alarmActuatorOn();
            alarmState = ALARM_WAITING;
            break;
        case ALARM_WAITING:
            delay(60000); // Wait 30 seconds before turning off the alarm
            alarmState = ALARM_OFF;
            break;
    }
}
```

Figure 17 — Alarm.c

Alarm.h

inc > Alarm.h > ...

```c
#ifndef ALARM_H_
#define ALARM_H_
/*Section 1: Include Files*/
#include "driver/Alarm_Actuator_Driver.h"
#include "High_Pressure_Detection.h"
/*Section 2: Macros*/
/*Section 3: Macro Functions*/
/*Section 4: Function Prototypes*/

 /**
 * @brief Finite State Machine tick function for alarm control
 */
void alarmfsmTick(void);

#endif /* ALARM_H_ */

```

Figure 18 — Alarm.h

FSM that monitors the high-pressure flag and controls the alarm for 60 seconds. Transitions: OFF → ON → WAITING (60s) → OFF.

→ Alarm Actuator Driver — Alarm_Actuator_Driver.c / .h

Alarm_Actuator_Driver.c

src > driver > C Alarm_Actuator_Driver.c > ALARM_ACTUATOR_STATE

```c
#include "driver/Alarm_Actuator_Driver.h"

static enum{
    ALARM_ACTUATOR_OFF = 0,
    ALARM_ACTUATOR_ON,
    ALARM_ACTUATOR_WAITING
}ALARM_ACTUATOR_STATE;

void alarmActuatorInit() {
    ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING;
}

void alarmActuatorOn() {
    if(ALARM_ACTUATOR_STATE == ALARM_ACTUATOR_WAITING) {
        ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_ON;
        setAlarmActuator(ALARM_ACTUATOR_ON);
        ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING;
    }
}

void alarmActuatorOff() {
    if(ALARM_ACTUATOR_STATE == ALARM_ACTUATOR_WAITING) {
        ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_OFF;
        setAlarmActuator(ALARM_ACTUATOR_OFF);
        ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING;
    }
}
```

Figure 19 — Alarm_Actuator_Driver.c

Alarm_Actuator_Driver.h

inc > driver > C Alarm_Actuator_Driver.h > ...

```c
#ifndef ALARM_ACTUATOR_DRIVER_H_
#define ALARM_ACTUATOR_DRIVER_H_

/*Section 1: Include Files*/
#include "driver.h"
/*Section 2: Macros*/
/*Section 3: Macro Functions*/
/*Section 4: Function Prototypes*/

/**
 * @brief Initialize the alarm actuator
 */
void alarmActuatorInit();

/**
 * @brief Turn the alarm actuator on
 */
void alarmActuatorOn();

/**
 * @brief Turn the alarm actuator off
 */
void alarmActuatorOff();
#endif // ALARM_ACTUATOR_DRIVER_H_
```

Figure 20 — Alarm_Actuator_Driver.h

Reactive driver that controls the physical alarm output. Only accepts commands when in WAITING state.

7. Startup & Linker

→ Linker Script — linkerscript.ld

linkerscript.ld

```
/*linkerscript.ld
** Eng.Adel Shata
 */

MEMORY
{
    flash(RX) : ORIGIN = 0x08000000, LENGTH = 128k
    sram(RWX) : ORIGIN = 0x20000000, LENGTH = 20k
}

SECTIONS
{
    .text :
    {

        *(.vectors*)
        *(.text*)
        *(.rodata*)
        . = ALIGN(4);
        _E_text = .;
    } > flash

    .data :
    {
        _S_data = .;
        *(.data*)
        . = ALIGN(4);
        _E_data = .;
    } > sram AT> flash

    .bss :
    {
        _S_bss = .;
        *(.bss*)
        . = ALIGN(4);
        _E_bss = .;
    } > sram

    _stack_top = . + 1000;
}
```

Figure 21 — Linker Script

Custom GNU LD linker script defining:

- FLASH: 0x08000000, 128 KB
- SRAM: 0x20000000, 20 KB
- Sections: .text (vectors + code + rodata → flash), .data (initialized → SRAM, load in flash), .bss (zero-initialized → SRAM)
- _stack_top = . + 1000

→ Startup File — startup.c (Page 1)

startup.c

```c
#include "Platform_Types.h"

extern uint32 _stack_top;
extern uint32 _E_text;
extern uint32 _S_data;
extern uint32 _E_data;
extern uint32 _S_bss;
extern uint32 _E_bss;

extern int main(void);

#define Reserved (uint32)0
void Reset_Handler(void);
void NMI_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void HardFault_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void MemoryManagement_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void BusFault_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void UsageFault_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void SVC_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void PendSV_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void SysTick_Handler(void) __attribute__((weak, alias("Reset_Handler")));
void ExternalInterruptsHandler(void) __attribute__((weak, alias("Reset_Handler")));

// initialize the vector table in flash memory
uint32 __attribute__((section(".vectors"))) vectors[] = {
    (uint32) &_stack_top,
    (uint32) &Reset_Handler,
    (uint32) &NMI_Handler,
    (uint32) &HardFault_Handler,
    (uint32) &MemoryManagement_Handler,
    (uint32) &BusFault_Handler,
    (uint32) &UsageFault_Handler,
    Reserved,
    Reserved,
    Reserved,
    Reserved,
    (uint32) &SVC_Handler,
    Reserved,
    Reserved,
    (uint32) &PendSV_Handler,
    (uint32) &SysTick_Handler,
    (uint32) &ExternalInterruptsHandler
};
```

Figure 22 — Startup — Vector Table

Custom C-based startup file. Defines the interrupt vector table with weak aliases for all fault handlers (default: reset on any fault).

→ Startup File — startup.c (Page 2)

```c

void Reset_Handler(void)
{
    void *src = NULL;
    void *dst = NULL;

    // Reallocate the .data section from flash to SRAM
    dst = (uint32 *)(&_S_data);
    src = (uint32 *)(&_E_text);
    uint32 _data_size = (uint32 *)(&_E_data) - (uint32 *)(&_S_data);
    for (uint32 i = 0; i < _data_size; i++){
        *((uint32 *)dst++) = *((uint32 *)src++);
    }

    // allocate the .data section in SRAM
    dst = (uint32 *)(&_S_bss);
    uint32 _bss_size = (uint32 *)(&_E_bss) - (uint32 *)(&_S_bss);
    for (uint32 i = 0; i < _bss_size; i++){
        *((uint32 *)dst++) = Reserved;
    }

    // branch to main function
    main();
}
```

Figure 23 — Startup — Reset Handler

Reset_Handler() — copies .data from FLASH to SRAM, zeros .bss, then calls main().

8. Build System

This project is built using a custom, reusable Makefile build engine maintained as a separate repository:

> **Universal_Make_Engine** ♥ https://github.com/Adel-Shata/Universal_Make_Engine.git
>
> Custom Makefile-based build engine for bare-metal ARM projects.
>
> Consult the linked repository for full documentation on configuration, usage, and available build options.

9. Simulation

The project was verified in Proteus VSM with a DIP-switch bank simulating the pressure sensor input and an LED simulating the alarm actuator output.

→ High Pressure Detected (pressure = 26, threshold = 20)

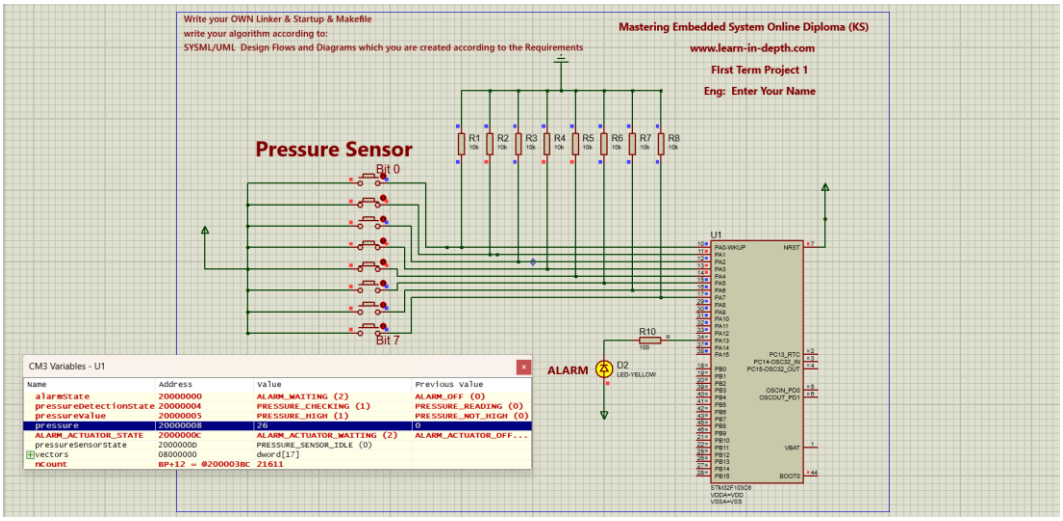

Figure 24 — Simulation — Alarm ON

When the pressure reading (26) exceeds the threshold (20):

- pressureValue = PRESSURE_HIGH (1)
- alarmState = ALARM_WAITING (2)
- ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_WAITING (2)
- Alarm LED (D2) is ON

→ Normal Pressure (pressure = 18, threshold = 20)

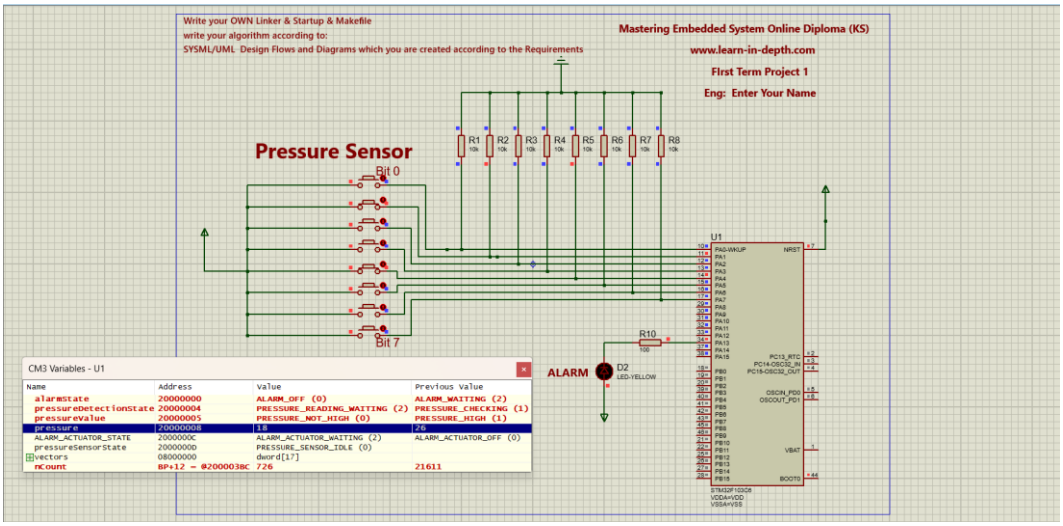

Figure 25 — Simulation — Alarm OFF

When the pressure reading (18) is below the threshold (20):

- pressureValue = PRESSURE_NOT_HIGH (0)
- alarmState = ALARM_OFF (0)
- ALARM_ACTUATOR_STATE = ALARM_ACTUATOR_OFF (0)
- Alarm LED (D2) is OFF

Debug Watch Variables

VARIABLE	MODULE	DESCRIPTION
pressureSensorState	Pressure_Sensor_Driver.c	PRESSURE_SENSOR_IDLE (0) / PRESSURE_SENSOR_READING
pressureDetectionState	High_Pressure_Detection.c	PRESSURE_READING (0) / PRESSURE_CHECKING (1) / PRESSURE_READING_WAITING (2)
alarmState	Alarm.c	ALARM_OFF (0) / ALARM_ON / ALARM_WAITING (2)
ALARM_ACTUATOR_STATE	Alarm_Actuator_Driver.c	ALARM_ACTUATOR_OFF (0) / ALARM_ACTUATOR_ON / ALARM_ACTUATOR_WAITING (2)
pressure	High_Pressure_Detection.c	Current raw pressure reading
pressureValue	High_Pressure_Detection.c	PRESSURE_NOT_HIGH (0) / PRESSURE_HIGH (1)

10. Repository Structure

```
Project_1_Pressure_Detection
    makefile
    toolchain.mk
    src
        main.c
        High_Pressure_Detection.c
        Alarm.c
        driver
            driver.c
            Pressure_Sensor_Driver.c
            Alarm_Actuator_Driver.c
    inc
        Platform_Types.h
        High_Pressure_Detection.h
        Alarm.h
        driver
            driver.h
            Pressure_Sensor_Driver.h
            Alarm_Actuator_Driver.h
    startup
        startup.c
    linker
        linkerscript.ld
    Screen_Shots
        UML/SysML diagrams
        Code screenshots
        Proteus simulation
    build
```